

HW1 – Group5

Calabrese Lorenzo (s345902) Calfapietro Flavia (s345862) Spedicato Giuseppe (s337116)

1. Whisper Evaluation

Model	Mem. (MB)	Acc. (%)	Lat. (s)
tiny	150MB	87.75%	2.88 ± 0.03
base	290MB	91.00%	7.84 ± 0.74
small	967MB	91.75%	22.26 ± 8.82
medium	3.06GB	91.88%	–
large	6.17GB	91.50%	–
large-v2	6.17GB	89.50%	–

Table 1: Evaluation results of Whisper model versions.

The results in Table 1 highlight the trade-offs between model size, memory usage, latency, and accuracy. Smaller models, such as **tiny** and **base**, require much less memory and run faster, but they have slightly lower transcription accuracy. Larger models, like **medium**, **large**, and **large-v2**, achieve higher accuracy, but their memory requirements and inference times are much higher. On a Raspberry Pi, only the **tiny**, **base**, and **small** models run smoothly, while larger models (medium, large, large-v2) exceed the available RAM and are therefore impractical for real-time execution. The Raspberry Pi has 2GB of RAM, the **medium** model needs over 3GB just for weights while the **large** and **large-v2** models more than 6GB, without considering additional memory for intermediate computations.

We can also observe that accuracy starts to decrease slightly from the large model onward. This effect can be explained by the fact that larger models are trained on broader and more complex, often multilingual, datasets. As a result, they may struggle to correctly recognize words or levels in the target language and sometimes transcribe a term in a different language. This multilingual interference can lead to a slight drop in overall accuracy, despite the model’s higher capacity and training size.

Even though Whisper models provide good transcription accuracy, they are not ideal for real-time, command-based voice interfaces. Whisper is designed for long audio streams rather than short, isolated keywords. This means it has higher latency and requires more memory than necessary for simple commands. For real-time keyword detection, a small, dedicated keyword-spotting model would be more suitable, providing fast responses and a low memory footprint.

2. Custom Training Pipeline Discussion

Pros and Cons

Replacing Whisper with a custom keyword spotting (KWS) model offers clear trade-offs. Whisper is ro-

bust to noise, accents, and varied speech, and its main advantage is that it is a speech-to-text trained model that can be used for any set of commands, across different domains, languages and question types. However, it is large, slow on edge devices, and overkill for detecting single commands. In practice, its inference latency is extremely high—around 2 seconds for the Tiny model, 7 seconds for the Small model and up to 22 seconds for the Base model—making it unsuitable for real-time applications. A custom KWS model, instead, is lightweight, fast and optimized for a limited set of keywords, allowing real-time operation on devices like a Raspberry Pi. Its drawbacks include the need for labeled data, struggling with noisy or complex audio and recognizing only the specific keywords it was trained to detect.

Training Pipeline Design

1. Collect Data: Collect audio clips for target keywords, unknown words and background noise. The dataset class loads the audio, stores waveforms and sampling rates and assigns integer labels for each class.

2. Preprocess Data: Convert all audio to mono 16 kHz signals and compute STFT, log-Mel spectrograms or MFCC features. Apply normalization and data augmentation to enhance robustness to real-world conditions.

3. Design a Model: Build a neural network architecture suited for on-device inference and final layer outputs probabilities for each class.

4. Train a Model: Use a suitable **loss function** to measure the discrepancy between predicted and true labels and an **optimizer** to iteratively adjust the model’s weights to minimize this loss.

5. Evaluate: Assess model accuracy on separate validation and test sets to ensure reliable evaluation.

6. Optimize: Apply pruning or quantization to reduce model size and inference time while maintaining acceptable accuracy. Optionally, use knowledge distillation from a larger model to improve the smaller network’s performance. Test the optimized model to ensure it maintains accuracy and performance after optimization.

7. Convert and Deploy Model: Convert the trained PyTorch model into an ONNX graph and deploy it directly on the edge device for efficient, real-time inference without cloud dependency.

8. Make Inferences: At runtime, the system processes incoming audio in short overlapping windows, extracts features and outputs class probabilities. A confidence threshold and smoothing logic make detection more reliable by requiring consistent, high-confidence predictions before triggering the “up” or “stop” commands.